

Intelligent Kubernetes Scheduling with Large Language Models

Xinyu Bai^{2,1}, Zhiqing Tang^{1*}, Zhi Yao¹, Yao Chen^{2,1}, Jianxiong Guo^{1,3}, Tian Wang^{1,4}, Weijia Jia^{1,3}

¹Institute of Artificial Intelligence and Future Networks, Beijing Normal University, China

²Faculty of Arts and Sciences, Beijing Normal University, China

³Guangdong Key Lab of AI and Multi-Modal Data Processing, Beijing Normal-Hong Kong Baptist University, China

⁴College of Computer and Data Science, Fuzhou University, China

{kioshiro, yaozhi, 202211079388}@mail.bnu.edu.cn, {zhiqingtang, jianxiongguo, tianwang, jiawj}@bnu.edu.cn

Abstract—The impressive natural language understanding and inference abilities of recent Large Language Models (LLMs) have enabled their use in scheduling tasks. Existing scheduling methods often neglect the integration of LLMs with schedulers. To fill in such gaps and address complex scheduling challenges in heterogeneous Kubernetes clusters, we propose two LLM-based approaches. Specifically: 1) *LLMScheduler*, an adaptive online scheduling framework in which the LLM actively participates in decision-making. It scores candidate nodes and selects the optimal one in real time for the clusters. 2) *HybridScheduler*, an offline intelligence injection framework that leverages LLMs to proactively generate strategic intents for the scoring function, informed by job profiles and node configurations. It then rapidly scores and selects nodes, similar to a traditional Kubernetes scheduler. 3) We validate our methods with large-scale simulations, demonstrating that *LLMScheduler* improves scheduling success rates by up to 7% on average compared to the best traditional scheduler, particularly under extreme cluster pressure. Moreover, *HybridScheduler* consistently outperforms all traditional heuristics in high-contention scenarios, while reliably maintaining millisecond-level decision latency. Our experimental results offer a quantitative foundation for designing future AI-driven systems that balance performance, latency, and cost.

Index Terms—Edge computing; Scheduling; Large Language Models

I. INTRODUCTION

Large Language Models (LLMs) like GPT-3 [1] and Llama-2 [2], built on the Transformer architecture [3], are transforming many fields with their powerful emergent abilities [4]. Their capacity for sophisticated reasoning and understanding of complex, high-level instructions presents a promising opportunity for optimizing resource management in large-scale computing clusters like Kubernetes. Unlike traditional schedulers that use fixed heuristics, LLMs can interpret ambiguous

objectives and make nuanced decisions in dynamic, heterogeneous environments. Despite their potential, the application of LLMs to the critical domain of Kubernetes system scheduling remains largely unexplored, creating a significant gap between modern AI capabilities and infrastructure needs.

To bridge this gap, an effective mechanism for the integration of LLMs and scheduling must be designed. Traditional schedulers are fast but lack the adaptability to handle the complex, multi-dimensional trade-offs in Kubernetes systems. Early AI-driven methods, such as those based on Reinforcement Learning (RL) [5]–[11], have showed promise but often require extensive, domain-specific training and complex reward function engineering. LLMs elegantly bypass this training bottleneck with their powerful zero-shot and few-shot reasoning capabilities. However, they introduce a new, fundamental challenge that stands as the central conflict of this paper: their **prohibitive inference latency** is fundamentally at odds with the millisecond-level decision window required for high-throughput production scheduling.

Therefore, to apply LLMs to the Kubernetes scheduling framework, the trade-off between performance and latency must be carefully considered and designed. The following challenges need to be addressed. *The first challenge is how to maximize the online decision-making capabilities of LLMs without taking decision latency into account.* Recent studies have begun to explore the potential of LLMs in other system-level decision-making tasks, such as assessing the quality of generated text and automating data annotation [12]–[14]. However, the domain of real-time scheduling presents unique latency constraints. In order to apply LLMs to scheduling, we first need to determine whether LLMs are actually capable of making decisions. To address this challenge, we propose the **LLMScheduler**, an adaptive online scheduling framework. At each scheduling cycle, this approach supplies the LLM with complete, real-time cluster data, enabling tactically optimal scheduling decisions. We hypothesize that this model, while impractical due to high latency, will establish the upper bound for scheduling quality.

Furthermore, the impracticality of the online approach presents our second research challenge: *How to distill an LLM's strategic insights into a lightweight, efficient policy*

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62302048 and Grant 62272050, in part by the Guangdong Key Lab of AI and Multi-modal Data Processing, Beijing Normal-Hong Kong Baptist University, Zhuhai under 2023-2024 Grants sponsored by Guangdong Provincial Department of Education, in part by Institute of Artificial Intelligence and Future Networks and Engineering Center of AI and Future Education, Guangdong Provincial Department of Science and Technology, China, in part by Zhuhai Science-Tech Innovation Bureau under Grant 2320004002772, and in part by the Interdisciplinary Intelligence Super Computer Center of Beijing Normal University at Zhuhai. (Corresponding author: Zhiqing Tang.)

suitable for production use? Recent advances such as AlphaEvo have used LLMs to act as a coding agent for scientific and algorithmic discovery that surpass those designed by humans [15]. However, using LLMs to generate scheduling algorithms is still uncharted territory. To address this, we propose the **HybridScheduler**, an offline intelligence-infused scheduling framework. In this method, the LLM acts as an offline strategic advisor, analyzing the global system properties beforehand to pre-compile a set of high-level strategic intents. A simple, deterministic local engine then executes these intents at millisecond-level speed. This approach delivers substantial AI performance gains while strictly meeting real-world latency constraints.

In this paper, we systematically investigate the performance-latency trade-off and address the challenges above. We have validated the proposed **LLMScheduler** and **HybridScheduler** through extensive simulations. The main contributions of this paper are as follows.

- 1) **Proposal of an Online-Adaptive Scheduling Framework:** To address the first challenge regarding the theoretical performance ceiling, we pioneer the design and implementation of the **LLMScheduler**. This paradigm provides a concrete methodology for leveraging an LLM as an omniscient oracle, establishing an upper bound for AI-driven scheduling quality.
- 2) **Proposal of an Offline-Infusion Scheduling Framework:** To tackle the second challenge of production feasibility, we design and implement the novel **HybridScheduler**. This offline intelligence infusion architecture offers a pragmatic pathway for pre-compiling an LLM’s strategic wisdom into a high-speed, low-latency local policy, directly resolving the performance-latency conflict.
- 3) **Systematic Experimental Validation:** We conduct the large-scale, systematic simulation experiments to quantitatively evaluate these two paradigms against a suite of traditional heuristics across a wide spectrum of resource contention levels. Our results validate the effectiveness of both approaches, providing a clear, data-driven map of the trade-off space for LLM-based scheduling.

The remainder of this paper is organized as follows. Section II reviews related work. Section III formalizes the scheduling problem and metrics. Section IV details our proposed methods. Section V presents the experimental setup and results. Finally, Section VI discusses our findings and concludes the paper.

II. RELATED WORK

Pioneering cluster schedulers like Google’s Borg [16] and Omega [17], along with Apache Mesos [18], laid the foundation for large-scale resource management but typically relied on hard-coded policies. Optimization-based schedulers like Firmament [19] attain global optimality by framing scheduling as a Minimum Cost Max Flow (MCMF) problem, but this approach demands that objectives be strictly represented as

costs. Our work shifts this paradigm from optimization-driven to intent-driven, reasoning directly from high-level objectives.

The modern Kubernetes Scheduling Framework [20] offers an extensible, pluggable architecture. This extensibility has enabled the creation of schedulers optimized for specific domains, such as Volcano for batch processing [21] and others designed for the unique constraints of edge computing environments [22], [23]. Our work extends this philosophy by introducing the higher-level concept of strategic intent.

Before LLMs, AI-driven scheduling mainly involved ML techniques requiring extensive training or complex reward design, such as QoS-aware prediction with collaborative filtering [9], Reinforcement Learning with MDPs [5], [8], or GNN-based resource forecasting [24]. In contrast, our LLM-based approach eliminates these requirements by utilizing the robust zero-shot reasoning of foundation models. Although recent work has begun to survey LLM for judgement [25], our study is among the first to systematically propose and contrast the online adaptation and offline infusion architectures for scheduling.

III. PROBLEM FORMALIZATION AND MODELING

To systematically evaluate LLM-based scheduling methods, we begin by formalizing the scheduling problem.

A. Problem Definition

We formulate scheduling as an online decision-making process where the scheduler receives a dynamic sequence of pending pods, $\mathbf{P} = \{p_1, p_2, \dots, p_k\}$. For each pod $p_i \in \mathbf{P}$, it must select the optimal node n_j from a set of heterogeneous nodes $\mathbf{N} = \{n_1, n_2, \dots, n_m\}$ for binding.

Constraints: Any valid scheduling decision must satisfy a series of hard constraints, which are strictly enforced during the Filter stage of our simulation. These include:

- **Resource Capacity:** The allocatable resources (e.g., CPU, Memory) of a candidate node n_j must meet or exceed the resource requests of the pending pod p_i .
- **Taints and Tolerations:** A pod p_i can only be scheduled to a node n_j if the pod’s tolerations can match all of the node’s taints. This is a core Kubernetes mechanism for node restriction.
- **Affinity and Anti-Affinity:** The scheduler must adhere to the `podAffinity` and `podAntiAffinity` rules specified in the pod’s definition, which control the co-location of pods.

Optimization Objective: After satisfying all hard constraints, the primary objective of the scheduler is to maximize the overall scheduling success rate. This is considered a global objective, as a short-sighted placement decision for one pod can lead to resource fragmentation, thereby negatively impacting the schedulability of subsequent pods. Secondary objectives, which contribute to the primary goal, include maximizing cluster resource utilization and maintaining load balance across the cluster. To comprehensively evaluate the schedulers, we define the following core metrics.

Scheduling Success Rate: This primary metric evaluates the scheduler's core function under resource constraints. It is the ratio of successfully scheduled pods (N_s) to the total number of pods attempted (N_t):

$$S = \frac{N_s}{N_t}. \quad (1)$$

Overall Cluster Resource Utilization: This metric reflects the efficiency of resource packing. We calculate it as the average of per-node utilization ratios to provide a granular measure:

$$U_{\text{cpu}} = \frac{1}{N} \sum_{j=1}^N \left(\frac{\sum_{i \in \mathbf{P}_j} R_{i,\text{cpu}}}{C_{j,\text{cpu}}} \right), \quad (2)$$

where N is the number of nodes, $\mathbf{P}_j$ is the set of pods on node j , $R_{i,\text{cpu}}$ is the CPU request of pod i , and $C_{j,\text{cpu}}$ is the node's capacity. Memory utilization U_{mem} is calculated identically.

Degree of Load Balancing: We use the standard deviation of node utilization to measure load distribution. A lower value indicates a more balanced cluster. It is defined as follows:

$$\sigma_U = \sqrt{\frac{1}{N} \sum_{j=1}^N (U_j - \bar{U})^2}, \quad (3)$$

where U_j is the utilization of node j , and $\bar{U}$ is the average utilization.

B. Model Assumptions and Simplifications

To highlight the key differences between scheduling paradigms, our simulation model is based on the following assumptions:

- 1) **Static Pod Lifecycle:** Once a pod is successfully scheduled, it occupies its requested resources permanently until the simulation ends. This simplifies the problem into a classic Online Bin Packing challenge, allowing us to evaluate performance based purely on final placement quality without the complexity of dynamic pod churn.
- 2) **Non-preemptive Scheduling:** Our model does not include a preemption mechanism. All pods are treated with equal priority, and a successfully scheduled pod will not be evicted.
- 3) **Closed-World Task Set:** Although pods arrive dynamically, the entire set originates from a pre-defined, finite workload. This ensures the reproducibility of our experiments.
- 4) **Simplified Resource Dimensions:** Our model primarily focuses on the core compute resources, CPU and Memory, as they are the most common constraints in general-purpose clusters.
- 5) **Static Cluster Environment:** The set of cluster nodes, N , is fixed during a single simulation run. We do not simulate dynamic node scaling or failures.

```

You are an expert Kubernetes scheduler. Your task is to
select the optimal node...

### Current Cluster State:
- node: hpc-node-0, available_cpu: 24.5, available_memory:
  64Gi, ...
... (and other nodes)

### Pending Pod Specification:
- name: pod-resource-hungry-3, resource_requests: {cpu:
  8.0, memory: 64Gi}, ...

### Instruction:
Based on the information above, which node is the best
choice? Return ONLY the node name.

```

Listing 1. Abridged example of a dynamic, tactical prompt for LLMScheduler.

IV. METHODOLOGY

To address the scheduling problem, we design two intelligent Kubernetes schedulers based on distinct philosophies, implemented within a lightweight, pluggable architecture inspired by the Kubernetes Scheduler Framework [20]. We categorize LLM applications in system control into two modes:

- **Online-Adaptive:** It uses the LLM as a decision oracle, consulted at each decision point for optimal quality.
- **Offline-Intelligence-Infusion:** It uses the LLM as an offline advisor, distilling its knowledge into a lightweight policy for fast local execution.

A. LLMScheduler

The **LLMScheduler** follows the Online-Adaptive paradigm and serves as our performance benchmark model, designed to explore the upper bounds of an LLM's scheduling capabilities.

Design Principles: The main principle is to achieve the highest possible scheduling quality, regardless of cost. We propose that by giving the LLM comprehensive, real-time context for each decision and removing the constraints of traditional heuristics, its advanced reasoning can consistently make near-optimal choices that outperform existing heuristic algorithms.

Workflow: The **LLMScheduler** follows a streamlined Observe-Query-Decide-Act loop:

- 1) **Observe:** When a pending pod enters the queue, the scheduler gathers the current cluster state, including each node's available resources and the labels and resource requests of existing pods.
- 2) **Query:** The scheduler dynamically generates an LLM-friendly prompt using real-time state and pending pod specifications, as shown in Listing 1. This prompt describes the problem context and directly requests the LLM to choose the optimal node for placement.
- 3) **Decide:** The scheduler sends the prompt to an external LLM service via API, which performs cloud inference and returns the most suitable node name.
- 4) **Act:** The scheduler parses and verifies the node name returned by the LLM, then binds the pod to that node.

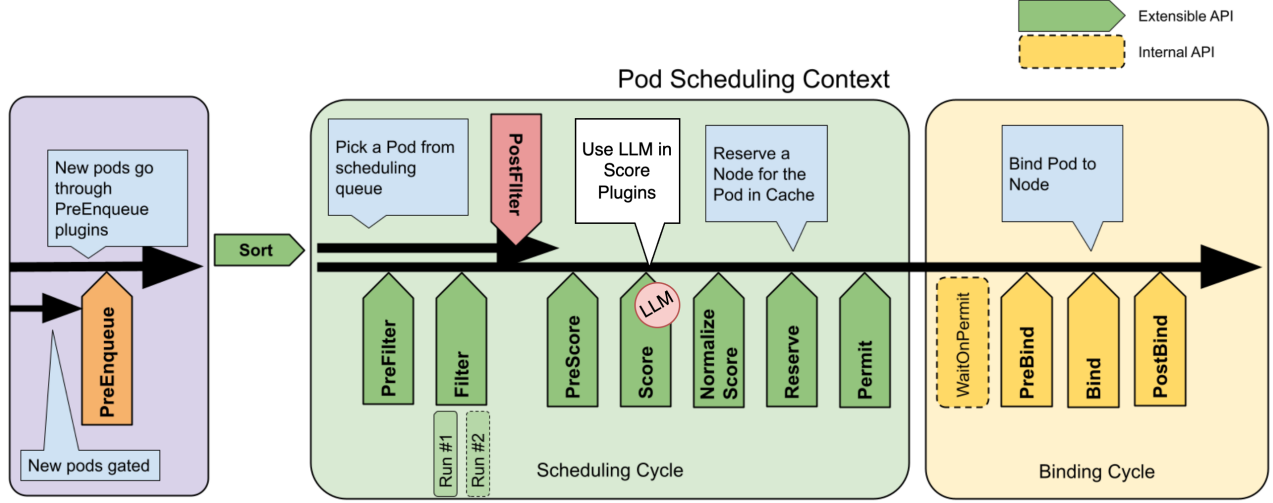


Fig. 1. The Kubernetes Scheduling Framework lifecycle and the integration point of our LLM-based scheduling logic. The framework is divided into several extension points (green). Our methods, particularly **LLMScheduler**, leverage the **Score** plugin to inject LLM-driven decision-making into the standard scheduling cycle, as highlighted by the callout.

B. HybridScheduler

To address the inherent high-latency and high-cost challenges of the **LLMScheduler**, we design the **HybridScheduler**, a fully production-oriented engineering solution. The design aims to leverage LLM’s strategic planning while maintaining the millisecond-level low latency of traditional schedulers. Its key innovation is a decoupled architecture that separates slow strategic planning from fast real-time scheduling, dividing the workflow into two independent stages.

a) *Offline Intent Generation*: In this stage, the LLM acts as a strategic advisor. An operator provides the LLM with the cluster’s macroscopic information (e.g., node hardware configurations, topology), expected workload types (e.g., a mix of online services and batch jobs), and high-level scheduling objectives (e.g., “prioritize packing compute-intensive tasks while ensuring fault isolation for critical services”). The LLM performs a one-time, deep deliberation to generate a global, static `strategic_intent` JSON file, as shown in Listing 2. This file is a high-level scheduling policy that maps different pod types to specific combinations and weights of plugins.

b) *Local High-Speed Execution*: At startup, the **HybridScheduler** loads the `strategic_intent` file. When processing incoming pods, it operates locally at high speed without network API calls. It retrieves the necessary scheduling plugins and scoring weights from the intent based on the pod’s profile, then efficiently runs local plugins through the standard Filter-Score process to make scheduling decisions.

HybridScheduler’s strategic flexibility is achieved through a modular plugin architecture. Two specialized plugins emulate the core scheduling strategies:

- The **SpreadingPlugin** aims to maximize fault tolerance by distributing pods across nodes. It penalizes nodes with

```
{
  "scheduling_strategy": {
    "pod_profile_mapping": {
      "cpu-intensive": "spreading",
      "memory-intensive": "spreading",
      "resource-hungry": "spreading",
      "pack": "spreading",
      "balanced": "spreading",
      "complementary": "spreading",
      "standard": "spreading",
      "edge": "spreading",
      "spread": "spreading",
      "cache-warmer": "spreading",
      "db-backup": "spreading"
    },
    "strategy_rationale": {
      "resource_pressure": 2.3,
      "priority": "scheduling_success_rate",
      "fallback_policy": "spreading",
      "node_utilization_goal": "avoid_conflicts",
      "constraint_handling": "strict_spreading"
    }
  }
}
```

Listing 2. Example of the strategic intents generated by LLM under a high-pressure scenario (Tension-5).

high resource utilization during scoring.

- The **PackingPlugin** seeks to optimize resource utilization by consolidating pods onto fewer nodes. It rewards nodes that would achieve higher density after pod allocation.

During the scoring phase, the simulator dynamically applies distinct strategies based on the `strategic_intent`, tailoring the behavior for different pods. This plugin-based design enables the coexistence and selective activation of competing optimization objectives within a unified framework.

V. EXPERIMENTS

This section outlines the experimental settings, workload configurations, and methodology used to evaluate the schedul-

ing paradigms.

A. Experimental Settings

1) *Simulator Core*: To enable a rigorous and reproducible evaluation, we develop a discrete-time, event-driven simulator. The simulator's architecture faithfully reproduces the Kubernetes scheduling flow by implementing a generic, two-stage **Filter-Score** pipeline. The core filtering engine (the `filter_node` method) supports not only resource capacity checks but also key scheduling rules such as `nodeSelector`, `taints/tolerations`, and `podAffinity/podAntiAffinity`. This high-fidelity design provides an equitable and realistic evaluation environment for all schedulers.

2) *Cluster Node Configuration*: We configure our experimental environment as a high-pressure, high-contention heterogeneous cluster to rigorously test each scheduler's decision-making. This cluster features diverse resource capacities, specialized hardware, varied topology, and different service tiers. It comprises 8 nodes, detailed in Table I. The composition is as follows:

- 1) **High-Performance Computing (HPC) Node**: This node provides powerful compute with 64 vCPUs and is the only `cpu-tier:premium` resource in the cluster, creating a critical bottleneck of scarce computational power.
- 2) **Memory-Intensive Nodes**: Each node provides 512 GB of memory and is located in a different data center (`zone:datacenter-a`, `zone:datacenter-b`) to evaluate High Availability (HA) strategies.
- 3) **Standard Nodes**: These serve as a general-purpose compute pool, also distributed across the two data centers.
- 4) **Edge Nodes**: Each node has limited resources (4 vCPU, 16 GB), is located in a separate edge zone, and is tainted with `node-type=edge:NoSchedule`.

3) *Workload Configuration*: We use a carefully designed synthetic workload with 9 distinct Pod profiles, classified by function and resource requirements.

a) *High-Demand & Intensive Profiles*: This group includes profiles like `cpu-intensive` (3 pods, 19.2 vCPU each), `memory-intensive` (5 pods, 205 GB each), `resource-hungry`, and `db-backup`, representing tasks that create significant pressure on specialized nodes.

b) *General-Purpose & Utility Profiles*: This group simulates the high volume of typical microservices and includes the most numerous profiles, `balanced` (16 pods) and `complementary` (13 pods), which test efficient packing on standard nodes.

c) *Constraint-Driven Profile*: This group includes the edge profile (6 pods), which has a specific toleration and can only be scheduled to a subset of nodes, testing the scheduler's ability to handle hard constraints.

Table II quantifies the total workload for each of the five tension levels, which are created by scaling both the number of pods and their resource requests.

4) *LLM Backend Configuration*: We employ the DeepseekV3-250324 API for its proficiency in non-Chain-of-Thought (non-CoT) reasoning.

B. Experimental Procedure

To ensure fairness and statistical significance, we employed a rigorous procedure. In each simulation, pods arrived dynamically based on their predefined `arrival_time`, creating a continuously evolving decision environment. To eliminate order bias, we randomly shuffled the submission sequence of the entire job set before each independent run. We conducted extensive repetitions for each scenario: **10,000 runs** for the fast, millisecond-level schedulers (all baselines and the **HybridScheduler**) and **20 runs** for the slower, seconds-level **LLMScheduler**, reflecting its higher time and computational costs.

C. Experimental Results and Analysis

1) *Overall Performance Trends Across Tension Levels*: Our experiments reveal a clear divergence in scheduler performance as resource pressure increases. At the low-contention level of Tension-1, all schedulers achieve near-100% success rates, establishing a clear baseline where algorithm choice has minimal impact (see Table III). However, as the cluster becomes saturated (Tension-3 and higher), the limitations of traditional heuristics become apparent, and the performance gap between different paradigms widens significantly, as illustrated across all subplots in Figure 2. Our subsequent analysis will focus primarily on the metrics from the most extreme scenario, Tension-5, to highlight these distinctions.

2) *Analysis of Scheduling Success Rate*: The scheduling success rate, our primary metric, underscores the superiority of the LLM-driven approaches in high-contention environments. As detailed in Table IV, the **LLMScheduler** establishes a clear performance ceiling, achieving a success rate of 50.14%. This represents a relative improvement of 3.03% and 5.19% over the `Spreading` and `Packing` baselines, respectively. Furthermore, our **HybridScheduler** also demonstrates a consistent advantage, with a success rate of 47.73% that reliably surpasses the best-performing baseline, `SpreadingScheduler` (47.11%), in all high-stress scenarios (Tension-3 to Tension-5).

3) *Analysis of Resource Utilization*: The resource utilization metrics, shown in Figure 2(b) and (c), explain the efficiency behind the success rates. The **LLMScheduler** exhibits exceptional packing efficiency, reaching a CPU utilization of 93.16% and memory utilization of 86.15% at Tension-5. This near-optimal placement prevents resource fragmentation, directly contributing to its higher success rate. In contrast, the other schedulers operate at a lower efficiency tier, with utilization hovering around 80% for `Hybrid` and `Spreading`, and dropping below 70% for the inefficient `Packing` scheduler.

4) *Analysis of Load Balancing*: The load balancing score (Figure 2(d)) reveals the core strategic differences and is key to understanding the final outcomes. Our **HybridScheduler** proves its design effectiveness by achieving a score of 1.38,

TABLE I
HIGH-PRESSURE CLUSTER NODE CONFIGURATION

Node Profile	Node Name	vCPU	Memory	Zone	Key Characteristics / Constraints
HPC	hpc-node-0	64	128 GB	datacenter-a	Only <code>cpu-tier:premium</code> node (Scarce Resource).
	memory-node-0	32	512 GB	datacenter-a	<code>memory-tier:high</code> .
Memory-Intensive	memory-node-1	32	512 GB	datacenter-b	Distributed across zones for HA tests.
	standard-node-0	16	64 GB	datacenter-a	General-purpose compute.
	standard-node-1	16	64 GB	datacenter-b	Distributed across zones for HA tests.
	standard-node-2	16	64 GB	datacenter-a	General-purpose compute.
Edge	edge-node-0	4	16 GB	edge-location-1	Taint: <code>node-type=edge:NoSchedule</code> .
	edge-node-1	4	16 GB	edge-location-2	Taint: <code>node-type=edge:NoSchedule</code> .
Total	8 Nodes	184	1376 GB	4 Zones	Heterogeneous, Multi-Constraint Environment.

TABLE II
DEFINITION OF RESOURCE TENSION LEVELS

Tension Level	Total Pods	Total CPU Req.	Total Mem Req.	Cluster Load (CPU / Mem)	Description
Tension-1	31	110.5 vCPU	618 GB	60.1% / 44.9%	Under-subscribed
Tension-2	44	157.0 vCPU	920 GB	85.3% / 66.9%	Approaching capacity
Tension-3	53	200.4 vCPU	1509 GB	108.9% / 109.7%	Saturated / Slightly over-subscribed
Tension-4	60	258.9 vCPU	1794 GB	140.7% / 130.4%	Highly over-subscribed
Tension-5	66	313.2 vCPU	2217 GB	170.2% / 161.1%	Extremely over-subscribed

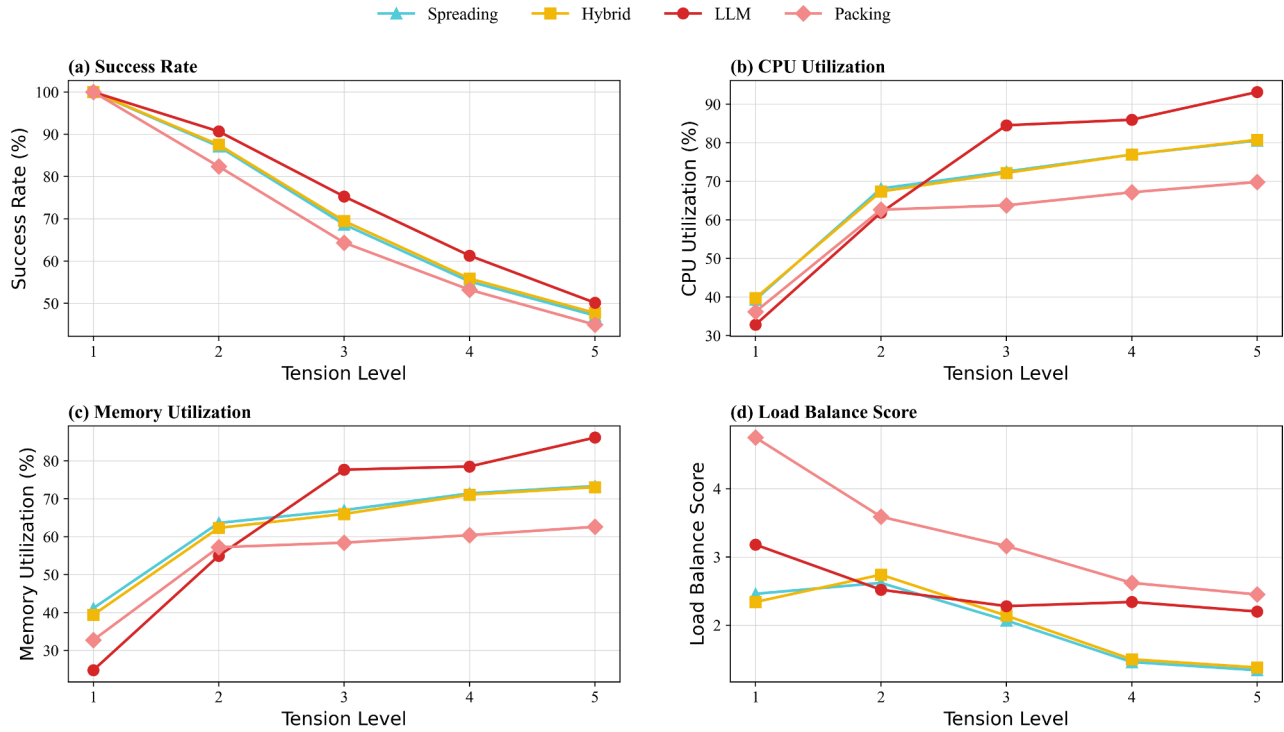


Fig. 2. Performance Trends of Schedulers Across Different Tension Levels. The four subplots show: (a) Scheduling success rate, (b) Average CPU utilization, (c) Average Memory utilization, and (d) Load balancing score (lower is better).

nearly identical to the best-in-class `SpreadingScheduler` (1.34). This demonstrates that its AI-generated offline strategy successfully inherits a robust, risk-averse placement policy. This balanced approach is critical for maintaining high success rates under pressure.

5) Insights on Baseline Performance: The load balancing metric also explains the failure of the worst-performing baseline. The `PackingScheduler` has the highest (worst) load balancing score of 2.45. Its greedy strategy of consolidating pods creates severe resource hotspots and leads to cluster-

TABLE III
PERFORMANCE BASELINE UNDER TENSION-1 (LOW LOAD)

Scheduler	Success Rate(%)	CPU Util.(%)	Mem Util.(%)	Load Balance
LLM	100.00	32.80	24.77	3.18
Hybrid	100.00	39.71	39.38	2.34
Spreading	100.00	39.30	41.13	2.46
Packing	99.99	36.17	32.73	4.75

TABLE IV
PERFORMANCE METRICS UNDER TENSION-5 PRESSURE

Scheduler	Success Rate (%)	CPU Util. (%)	Mem Util. (%)	Load Balance
LLM	50.14	93.16	86.15	2.20
Hybrid	47.73	80.77	73.06	1.38
Spreading	47.11	80.58	73.36	1.34
Packing	44.95	69.82	62.61	2.45

wide resource fragmentation. This leads to a low success rate, demonstrating the fragility of this strategy under dynamic workloads and underscoring the superiority of the more balanced `SpreadingScheduler` and our `HybridScheduler`.

VI. CONCLUSION

This paper addresses the central challenge of applying LLMs to critical systems: balancing theoretical optimality and practical feasibility. Our experiments demonstrate that the online approach (`LLMScheduler`) sets the upper limit for performance, while our offline-infusion model (`HybridScheduler`) offers substantial performance gains within latency constraints. However, our simulator cannot fully capture real-world complexities, making validation on a physical cluster an essential next step. Future research may consider meta-schedulers or adaptive hybrid models. Ultimately, integrating AI into large-scale, mission-critical systems requires carefully balancing performance, latency, and cost, and the hybrid architecture offers a promising solution.

REFERENCES

- [1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 1877–1901.
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer *et al.*, "Llama 2: Open foundation and fine-tuned chat models," 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, and o. Dhariwal, "Language models are few-shot learners," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS '20. Red Hook, NY, USA: Curran Associates Inc., 2020.
- [5] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*, ser. HotNets '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 50–56. [Online]. Available: <https://doi.org/10.1145/3005745.3005750>
- [6] Z. Tang, F. Mou, J. Lou, W. Jia, Y. Wu, and W. Zhao, "Multi-user layer-aware online container migration in edge-assisted vehicular networks," *IEEE/ACM Transactions on Networking*, vol. 32, no. 2, pp. 1807–1822, 2023.
- [7] H. Cui, Z. Tang, J. Lou, W. Jia, and W. Zhao, "Latency-aware container scheduling in edge cluster upgrades: A deep reinforcement learning approach," *IEEE Transactions on Services Computing*, vol. 17, no. 5, pp. 2530–2543, 2024.
- [8] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning Scheduling Algorithms for Data Processing Clusters," in *Proceedings of the ACM SIGCOMM 2019 Conference (SIGCOMM '19)*. New York, NY, USA: ACM, 2019, pp. 371–384.
- [9] C. Delimitrou and C. Kozyrakis, "Paragon: Qos-aware scheduling for heterogeneous datacenters," in *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 77–88. [Online]. Available: <https://doi.org/10.1145/2451116.2451125>
- [10] Z. Tang, J. Lou, and W. Jia, "Layer dependency-aware learning scheduling algorithms for containers in mobile edge computing," *IEEE Transactions on Mobile Computing*, vol. 22, no. 6, pp. 3444–3459, 2022.
- [11] H. Cui, Z. Tang, Y. Wu, and W. Jia, "Layer-aware cost-effective container updates with edge-cloud collaboration in edge computing," *IEEE Transactions on Mobile Computing*, 2025.
- [12] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu *et al.*, "A survey on llm-as-a-judge," *arXiv preprint arXiv:2411.15594*, 2024.
- [13] Q. He, Z. Feng, H. Fang, X. Wang, L. Zhao, K. Yu, and K.-K. R. Choo, "Blockchain-based edge computing service with dynamic entry and exit mechanism," *IEEE Transactions on Mobile Computing*, 2025.
- [14] Q. He, Z. Feng, Z. Chen, T. Nan, K. Li, H. Shen, K. Yu, and X. Wang, "Low-cost data offloading strategy with deep reinforcement learning for smart healthcare system," *IEEE Transactions on Services Computing*, 2024.
- [15] A. Novikov, N. Vü, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. Ruiz, A. Mehrabian *et al.*, "Alphaevolve: A coding agent for scientific and algorithmic discovery," *arXiv preprint arXiv:2506.13131*, 2025.
- [16] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at google with borg," in *Proceedings of the Tenth European Conference on Computer Systems*, ser. EuroSys '15. New York, NY, USA: Association for Computing Machinery, 2015. [Online]. Available: <https://doi.org/10.1145/2741948.2741964>
- [17] M. Schwarzkopf, A. Konwinski, M. Abd-El-Malek, and J. Wilkes, "Omega: flexible, scalable schedulers for large compute clusters," in *Proceedings of the 8th ACM European Conference on Computer Systems*, ser. EuroSys '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 351–364. [Online]. Available: <https://doi.org/10.1145/2465351.2465386>
- [18] B. Hindman, A. Konwinski, M. Zaharia, A. Ghodsi, A. D. Joseph, R. Katz, S. Shenker, and I. Stoica, "Mesos: a platform for fine-grained resource sharing in the data center," in *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation*, ser. NSDI'11. USA: USENIX Association, 2011, p. 295–308.
- [19] I. Gog, M. Schwarzkopf, A. Gleave, R. N. M. Watson, and S. Hand, "Firmament: fast, centralized cluster scheduling at scale," in *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'16. USA: USENIX Association, 2016, p. 99–115.
- [20] The Kubernetes Authors, "Scheduling Framework," 2024, kubernetes Documentation. Accessed: June 27, 2025. <https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/>.
- [21] Volcano Authors, "Volcano: Cloud native batch scheduling system for compute-intensive workloads," 2024, cNCF Sandbox Project. Accessed: June 27, 2025. <https://volcano.sh/>.
- [22] W. Peng, Z. Tang, J. Guo, J. Lou, T. Wang, and W. Jia, "LR2Scheduler: Layer-aware, Resource-balanced, and Request-adaptive

- Container Scheduling for Edge Computing,” *CCF Transactions on Pervasive Computing and Interaction (TPCI)*, pp. 1–15, 2025.
- [23] Z. Tang, W. Peng, J. Guo, J. Lou, H. Cui, T. Wang, Y. Wu, and W. Jia, “LRScheduler: A Layer-aware and Resource-adaptive Container Scheduler in Edge Computing,” in *2024 20th International Conference on Mobility, Sensing and Networking (MSN)*. IEEE, 2024.
 - [24] Z. Wang, B. He, K. Li, D. Rajan, and K.-L. Tan, “Gras: A Graph-based Resource-Aware Scheduler for Deep Learning,” in *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2021, pp. 831–840.
 - [25] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Y. Wang, W. Gao, L. Ni, and J. Guo, “A survey on llm-as-a-judge,” 2025. [Online]. Available: <https://arxiv.org/abs/2411.15594>